Project 2 - Parser

Deadline：10 月 21 日 23:59

不接受任何理由的迟交，迟交原则上不予受理。

成绩以最后一次提交为准。

每个小组仅需一位同学提交。

1 项目要求

在 Project 1 中，你已经完成了 Splc 的词法分析。在 Project 2 中，我们将进一步完成语法分析、Parse Tree 遍历以及基础的语法错误恢复。
你将在 Project 2 中完成以下任务：

任务一： 基于已给出的语法框架，补全 `expression` 规则，并正确处理操作符的优先级与结合性；

任务二： 使用 ANTLR 的 **Visitor 模式**，实现对本项目所定义的常量表达式（`constexpr`）的识别与求值；

任务三： 基于 ANTLR 的 `DefaultErrorStrategy`，实现 Missing Symbol 与 Extraneous Symbol 的错误报告，并针对指定的语法错误，使用 **Error Production** 与 **Grammar Action** 完成局部错误恢复。

注意：
本 Project 固定使用 Visitor 遍历 Parse Tree。框架会使用：

```
-visitor -no-listener
```

生成 ANTLR 代码。
需要注意的是，`-no-listener` 表示不生成 Parse Tree Listener。框架中的 `Project2ErrorListener` 是 ANTLR 的错误监听器，与 Parse Tree Listener 并不是同一种机制。

2 任务一：补全表达式语法规则

本节需要你根据第 2.9 节给出的产生式及操作符的优先级和结合性，补全 `Splc.g4` 中的 `expression` 规则。第 2.1～2.8 节中的 Parser Rules 已由 Starter Code 提供，这些小节仅用于介绍已有的语法规则。

> **注意：**
>
> - **中文标签与 Parser Rule Name：**本节标题和表格中的“程序”、“结构体定义”、“函数调用”等中文词仅作阅读标签。实际规则名见小节标题中的代码字体或完整产生式的左侧，例如 `program`、`structDef`、`expression`。表格中的“函数调用”等是 `expression` 的表达式形式，并非独立规则。
> - **不要修改规则名：**Starter Code 中已有的规则名是 Framework 和 Visitor 使用的接口。除非本文档明确要求，**请不要修改这些规则名**，否则可能无法通过评测。

2.1 程序（program）

```
program
    : globalDef* EOF
    ;
```

规则概述：一个完整程序由零个或多个 `globalDef` 组成，并最终到达 `EOF`。

2.2 全局定义（globalDef）

```
globalDef
    : structDef
    | functionDef
    | globalVarDef
    ;
```

规则概述：全局定义可以是结构体定义（`structDef`）、函数定义（`functionDef`）或全局变量定义（`globalVarDef`）。

2.3 类型（type）

```
type
    : INT
    | STRUCT Identifier
    ;
```

规则概述：类型可以是 `int`，也可以是 `struct` 后接标识符。

例如：

```
int a;
struct Point p;
```

2.4 结构体定义（structDef）

```
structDef
    : STRUCT Identifier LBRACE fieldDef+ RBRACE SEMI
    ;
```

规则概述：结构体定义至少包含一个字段（`fieldDef+`），并以分号结束。

其中，字段定义使用：

```
fieldDef
    : variableDecl SEMI
    ;
```

例如：

```
struct Point {
    int x;
    int y;
};
```

2.5 变量声明（variableDecl）

```
variableDecl
    : type Identifier arrayDimension*
    ;
```

规则概述：变量声明由类型、标识符和零个或多个数组维度组成。数组长度在本 Project 的语法层面使用 `Number`。

数组维度使用：

```
arrayDimension
    : LBRACK Number RBRACK
    ;
```

全局变量定义使用：

```
globalVarDef
    : variableDecl SEMI
    ;
```

局部变量定义使用：

```
localVarDef
    : variableDecl (ASSIGN expression)? SEMI
    ;
```

例如：

```
int a;
int b[10];
int matrix[2][3];
struct Point points[8];
```

2.6 函数定义（functionDef）

```
functionDef
    : type Identifier LPAREN parameterList? RPAREN block
    ;
```

规则概述：函数定义由返回类型、函数名、可选参数列表和代码块组成。

参数列表使用：

```
parameterList
    : parameter (COMMA parameter)*
    ;
```

单个形式参数使用：

```
parameter
    : variableDecl
    ;
```

例如：

```
int add(int a, int b) {
    return a + b;
}

int get(int a[10]) {
    return a[0];
}
```

2.7 代码块（block）

```
block
    : LBRACE statement* RBRACE
    ;
```

规则概述：代码块由花括号包围零个或多个 `statement`；在本 Project 中，`if`、`else` 和 `while` 的主体均使用 `block`。

2.8 语句（statement）

```
statement
    : block
    | localVarDef
    | ifStatement
    | whileStatement
    | returnStatement
    | expressionStatement
    ;
```

规则概述：语句可以是代码块、局部变量定义、`if` 语句、`while` 语句、`return` 语句或表达式语句。

其中，`if`、`while`、`return` 和表达式语句的产生式如下。

2.8.1 If 语句（ifStatement）

```
ifStatement
    : IF LPAREN expression RPAREN
      block
      (ELSE block)?
    ;
```

2.8.2 While 语句（whileStatement）

```
whileStatement
    : WHILE LPAREN expression RPAREN
      block
    ;
```

2.8.3 Return 语句（returnStatement）

```
returnStatement
    : RETURN expression SEMI
    ;
```

2.8.4 表达式语句（expressionStatement）

```
expressionStatement
    : expression SEMI
    ;
```

2.9 表达式（expression）

规则概述： `expression` 是本 Project 中需要补全的主要 Parser Rule。本节给出它需要支持的全部右侧产生式，并在后文说明优先级与结合性。

需要注意：

下面为了便于阅读，会将不同形式的表达式**分组列出**。
但是这些并不是多个独立的 Parser Rule。
例如，“函数调用”,“数组访问”,“一元负号”以及“加法”等都只是 `expression` 的不同产生式形式。
最终，你仍然只需要完成一个：

```
expression
    : ...
    ;
```

Starter Code 不要求你另外创建 `primaryExpression`、`binaryExpression`、`unaryExpression` 等规则。

你需要做的主要工作是：

- 将下面给出的所有右侧产生式正确组织到 `expression` 中；
- 根据后文给出的优先级安排这些产生式的顺序；
- 正确处理左结合与右结合。

2.9.1 基本形式

下面这些都是 expression 的产生式：

表达式形式	右侧产生式
整数	Number
标识符	Identifier
括号表达式	LPAREN expression RPAREN

2.9.2 后缀形式

下面这些同样都是 expression 的产生式：

表达式形式	右侧产生式
函数调用	Identifier LPAREN argumentList? RPAREN
数组访问	expression LBRACK expression RBRACK
结构体成员访问	expression DOT Identifier

函数实参列表使用一个真实的辅助 Parser Rule：

```
argumentList
    : expression (COMMA expression)*
    ;
```

这里需要注意：表格中的“函数调用”只是 expression 的一种右侧产生式；只有 argumentList 是独立的辅助 Parser Rule。

2.9.3 前缀一元操作符

下面这些也都是 expression 的产生式：

表达式形式	右侧产生式
一元正号	PLUS expression
一元负号	MINUS expression
逻辑非	NOT expression

2.9.4 二元操作符

下面这些同样全部属于 expression：

表达式形式	右侧产生式
幂运算	expression POW expression
乘法	expression STAR expression
除法	expression DIV expression
取模	expression MOD expression
加法	expression PLUS expression
减法	expression MINUS expression

小于	`expression LT expression`
小于等于	`expression LE expression`
大于	`expression GT expression`
大于等于	`expression GE expression`
等于	`expression EQ expression`
不等于	`expression NEQ expression`
逻辑与	`expression AND expression`
逻辑或	`expression OR expression`
赋值	`expression ASSIGN expression`

以上四组内容全部都是同一个 `expression` Parser Rule 的右侧产生式。

> 你不需要自行设计新的表达式语法形式，也不要删除或增加上面列出的产生式。你需要做的是根据下面给出的优先级和结合性，将这些产生式正确组织到 `expression` Rule 中。

2.9.5 优先级与结合性

ANTLR4 支持直接左递归的表达式规则。
在本 Project 中，你需要按照下面的优先级，将前面已经给出的右侧产生式组织到 `expression` 中。
从高到低，优先级为：

1. 函数调用、数组访问、结构体成员访问；
2. `POW`；
3. 一元 `PLUS`、`MINUS`、`NOT`；
4. `STAR`、`DIV`、`MOD`；
5. `PLUS`、`MINUS`；
6. `LT`、`LE`、`GT`、`GE`；
7. `EQ`、`NEQ`；
8. `AND`；
9. `OR`；
10. `ASSIGN`。

其中：

- `POW` 为右结合；
- `ASSIGN` 为右结合；
- 其余二元操作符均按通常方式左结合。

例如，`a = b = 1` 应按照 `a = (b = 1)` 解析；
同样，`2 ^ 3 ^ 4` 应按照 `2 ^ (3 ^ 4)` 解析。

3 任务二：常量表达式求值

本部分的目标是使用 Visitor 遍历 Parse Tree，识别常量表达式并计算其值。

3.1 常量表达式的定义

本 Project 中，满足以下条件的表达式属于**常量表达式**（`constexpr`）：

- ✓ **整数常量**属于常量表达式。
- ✓ **括号表达式**在括号内的表达式为常量表达式时，也属于常量表达式。
- ✓ **一元正号或一元负号表达式**在操作数为常量表达式时，也属于常量表达式。
- ✓ **加、减、乘、除或取模表达式**在两个操作数均为常量表达式时，也属于常量表达式。

是常量表达式的例子：

- `42`
- `1 + 2 * 3`
- `-(10 - 4) / 2`
- `20 % 6 + 1`

除此之外，其他形式的表达式均不属于本 Project 定义的常量表达式，例如：

- ✗ 变量；
- ✗ 函数调用；
- ✗ 数组访问；
- ✗ 结构体成员访问；
- ✗ 逻辑非 `NOT`；
- ✗ 幂运算 `POW`；
- ✗ 关系运算、相等运算和逻辑运算；
- ✗ 赋值表达式。

不是常量表达式的例子：

- `x`
- `x + 2`
- `foo(1, 2)`
- `a[0] + 3`
- `!0`
- `2 ^ 3`
- `1 < 2`

评测不会要求你处理整数溢出，也不会出现除数为 0 或模数为 0 的 `constexpr`。

3.2 常量表达式的求值

本节只处理局部变量定义中的初始化表达式：判断 ASSIGN 右侧的**完整表达式**是否为 constexpr；若是，就将表达式文本及其值加入结果列表。对应的语法规则为：

```
localVarDef
    : variableDecl (ASSIGN expression)? SEMI
    ;
```

例如，在下面的局部变量定义中，完整初始化表达式是 constexpr：

```
int a = 1 + 2 * 3;
```

需要记录表达式文本 1 + 2 * 3 及其计算结果 7。

注意，以下情况不记录结果：

1. **不单独记录常量子表达式。**

   若完整初始化表达式不是 constexpr，则不记录结果；即使其中有常量子表达式，也不单独记录。例如：

   ```
   int b = (1 + 2) * x;
   ```

   这里的完整初始化表达式含有变量 x，整体不是 constexpr；其中的 1 + 2 虽然是常量表达式，也**不会被单独记录**。

2. **普通赋值表达式不在处理范围内。**

   普通赋值表达式不在本节处理范围内。例如：

   ```
   a = 1 + 2;
   ```

   这不是局部变量定义中的初始化表达式，因此不生成常量表达式结果。

3.3 实现要求

在 ConstExprVisitor.java 中，Starter Code 已经提供：

```
protected final void addResult(
        String expression,
        int value)
```

以及结果列表的管理逻辑。
你需要完成：

- 找到所有带初始化表达式的 localVarDef；
- 判断其完整右值是否为 constexpr；
- 对 constexpr 递归求值；
- 使用 addResult(...) 保存表达式文本和值。

不要自行打印结果。输出格式由 framework 统一处理。

4 任务三：语法错误报告与恢复

本部分的目标是处理指定范围内的语法错误，完成错误报告与必要的局部恢复。

4.1 错误类型与测试约束

本 Project 中需要处理两种错误：

- **Missing Symbol**：缺失一个本应存在的 Token；
- **Extraneous Symbol**：存在一个多余 Token。

为了保证错误恢复行为可确定，本 Project 的语法错误测试满足：

- 每个测试用例最多只包含一处源代码级语法错误；
- 只测试本文档明确列出的错误形式；
- 不要求实现任意错误输入下的完整错误恢复。

4.2 需要处理的错误形式

本部分只处理下表列出的错误，并按后文的规范报告 Token 类型与行号：

错误	示例形式
缺少 SEMI	`return expression RBRACE`
多余 RPAREN	`return expression RPAREN SEMI`
数组声明缺少 RBRACK	`int a[10;`
数组访问缺少 RBRACK	`return a[1;`
函数调用缺少 RPAREN	`foo(1, 2;`
If 条件缺少 RPAREN	`if (expression LBRACE ...`
While 条件缺少 RPAREN	`while (expression LBRACE ...`
函数参数列表末尾缺少 RPAREN	`int f(int a LBRACE ...`
函数形式参数之间缺少 COMMA	`int f(int a int b)`

例如，函数形式参数列表中缺少 COMMA 时：

正确输入：

```
int add(int a, int b) {
    return a + b;
}
```

错误输入：

```
int add(int a int b) {
    return a + b;
}
```

该错误必须报告为：

```
Missing Symbol 'COMMA' at Line <LINE>
```

4.3 错误行号的定义

本 Project 中所有输出行号均**从 0 开始计数**（0-indexed）。

4.3.1 Missing Symbol

对于 Missing Symbol，其行号定义为：

> Parser 判断发生 Token 缺失时，**当前尚未消费的 Lookahead Token 所在的行号**。

例如：

```
if (a < b
{
    return 1;
}
```

当 Parser 判断缺少 `RPAREN` 时，当前 Lookahead Token 是 `LBRACE`，因此错误行号取 `LBRACE` 所在行。

4.3.2 Extraneous Symbol

对于 Extraneous Symbol，其行号定义为：

> 该多余 Token 本身所在的行号。

> **注意：**
> ANTLR 的 `Token.getLine()` 返回从 1 开始计数的行号，报告时需要减 1，转换为本项目要求的从 0 开始计数的行号。

4.4 错误报告的输出格式

最终输出由 Framework 统一生成，以下格式用于说明预期结果，无需自行拼接错误字符串或打印错误报告。
Missing Symbol：

```
Missing Symbol '<TOKEN_NAME>' at Line <LINE>
```

例如：

```
Missing Symbol 'SEMI' at Line 3
```

Extraneous Symbol：

```
Extraneous Symbol '<TOKEN_NAME>' at Line <LINE>
```

例如：

```
Extraneous Symbol 'RPAREN' at Line 2
```

Token 名称使用 ANTLR Vocabulary 中的 Symbolic Name，例如 `SEMI`、`RPAREN`、`RBRACK`。

4.5 默认错误报告的实现要求

ANTLR 的 `DefaultErrorStrategy` 已实现通用的局部错误恢复，包括 Single-Token Insertion 与 Single-Token Deletion。本 Project 不要求你重新实现恢复算法本身，只需补全恢复过程中的错误报告逻辑。
你需要在 `Project2ErrorStrategy.java` 中补全以下两个方法：

```
protected void reportMissingToken(Parser recognizer)
protected void reportUnwantedToken(Parser recognizer)
```

Starter Code 已在这两个方法中提供：

```
if (inErrorRecoveryMode(recognizer)) {
    return;
}
beginErrorCondition(recognizer);
// TODO
```

其中，`inErrorRecoveryMode(...)` 用于避免同一次错误恢复中的重复报告。
`beginErrorCondition(...)` 将当前错误策略标记为已进入错误恢复状态。
在此基础上，你需要完成：

- 从 `Parser` 和 `DefaultErrorStrategy` 提供的信息中确定缺失或多余的 Token 类型；
- 按前文的行号定义确定错误行号；
- 调用 Framework 提供的对应报告接口。

报告接口如下：

```
reporter.reportMissing(int tokenType, int line);
reporter.reportExtraneous(int tokenType, int line);
```

> **注意：**
> 应使用 ANTLR 提供的结构化 Parser / Token 信息判断错误类型与 Token，不要解析 ANTLR 默认产生的英文错误信息字符串。

4.6 特定错误恢复的实现要求

对于默认恢复机制无法按要求处理的特定错误结构，你需要在 `Splc.g4` 的相应语法规则中加入 Error Production，并使用 Grammar Action 报告错误。
Starter Grammar 已提供：

```
reportMissing(int tokenType, int line)
currentLine()
```

你需要在错误产生式中使用 `reportMissing(...)` 报告缺失的 Token 类型与行号。其中，`currentLine()` 返回当前 Lookahead Token 的行号，已经从 0 开始计数，可以直接使用。

> **注意：**
> 仅为前文列出的错误形式补充必要的 Error Production；默认恢复机制能够按要求处理的情况无需重复添加规则。

5 初始代码

Starter Code 中已经提供完整的项目框架。
代码仓库:`https://github.com/sqlab-sustech/CS323-Compilers-2026F-Projects`。请使用仓库的 `project2` 分支。
主要目录结构如下:

```
Project2/
|-- Makefile
|-- Splc.g4
|-- libs/
|   `-- .gitkeep
|-- src/main/java/
|   |-- Main.java
|   |-- ConstExprVisitor.java
|   `-- framework/project2/
|       |-- ConstExprResult.java
|       |-- ErrorKind.java
|       |-- ErrorReporter.java
|       |-- Grader.java
|       |-- Project2ErrorListener.java
|       |-- Project2ErrorStrategy.java
|       `-- SyntaxDiagnostic.java
`-- testcases/project2/
    |-- in/
    `-- out/
```

请将:

```
antlr-4.13.2-complete.jar
```

放入:

```
libs/
```

5.1 你需要修改的文件

你只需要修改以下三个文件，各文件与前面任务的对应关系如下。

5.1.1 `Splc.g4`

该文件对应**任务一**，以及**任务三**中的特定错误恢复:

- **任务一**: 补全 `expression` 规则，正确处理操作符的优先级与结合性;
- **任务三**: 为形式参数之间缺少 `COMMA` 添加指定 Error Production，并使用 Grammar Action 报告错误。

5.1.2 `ConstExprVisitor.java`

该文件对应**任务二：常量表达式求值**。你需要：

- 找到局部变量初始化中的完整右值；
- 判断其是否为 `constexpr`；
- 完成递归求值；
- 使用 `addResult(...)` 保存结果。

5.1.3 `Project2ErrorStrategy.java`

该文件对应**任务三**中的默认错误报告。你需要：

- 实现 Missing Symbol 的报告；
- 实现 Extraneous Symbol 的报告；
- 不重新实现 ANTLR 的 Single-Token Insertion / Deletion。

> 最终只需提交上述三个文件到 OJ。提交方式见第 8节。

6 程序输出

6.1 语法正确的程序

对于语法正确的程序，framework 会依次输出：

- Parse Tree；
- 所有符合要求的 `constexpr` 及其计算结果。

Parse Tree 使用：

```
tree.toStringTree(parser)
```

常量表达式输出格式为：

```
Expression: `<EXPRESSION>` evaluates to <VALUE>
```

例如：

```
Expression: `1+2*3` evaluates to 7
```

表达式文本来自 Parse Tree，因此其中不会保留被 Lexer 跳过的空白和注释。

6.2 存在语法错误的程序

对于本 Project 要求处理的错误，如果成功检测到语法错误，则程序只输出唯一的错误诊断，不输出 Parse Tree 或 `constexpr` 结果。

7 编译与测试

7.1 编译

执行：

```
make
```

或：

```
make build
```

Makefile 会依次：

- 使用 ANTLR 生成 Lexer、Parser 与 Visitor；
- 使用 `javac` 编译 Java 文件；
- 生成 `target/project2.jar`。

7.2 运行

你可以通过标准输入运行：

```
make run < testcase.splc
```

7.3 公开测试

执行：

```
make test
```

Makefile 会自动遍历：

```
testcases/project2/in/*.splc
```

并与：

```
testcases/project2/out/*.out
```

中的同名输出比较。
例如：

```
testcases/project2/in/003_constexpr.splc
```

对应：

```
testcases/project2/out/003_constexpr.out
```

测试会运行全部公开样例。例如：

```
[PASS] 001_basic_global
[PASS] 002_struct_array

[FAIL] 003_constexpr
----------------------------------------

...
----------------------------------------

Result: 2 / 3 passed
```

公开测试并不覆盖所有情况，你需要自行设计额外测试验证你的实现。

8 提交

执行：

```
make handin
```

所有公开测试通过后，将生成：

```
target/project2.zip
```

该 ZIP 中只包含：

```
Splc.g4
ConstExprVisitor.java
Project2ErrorStrategy.java
```

三个文件均位于 ZIP 根目录。
请直接将该 ZIP 上传至 Hydro OJ。

> 不要自行重新组织 ZIP 内目录，也不要加入额外文件。OJ 会检查提交包中的文件名和目录结构。

9 评分

本 Project 满分 100 分。
最终评测使用 20 个隐藏测试样例，每个测试样例 5 分。
测试内容从基础到综合逐步增加，覆盖：

- 基本 Parser Rules；
- 表达式优先级与结合性；
- 函数、数组、结构体与成员访问；
- 控制流与复杂表达式组合；
- `constexpr` 的识别与求值；
- ANTLR 默认 Single-Token Recovery 的错误报告；

- Missing Symbol 与 Extraneous Symbol 的 Token 类型及行号；
- 指定的 Missing `COMMA` Error Production。

对于语法正确的测试：

- Parse Tree 必须与标准答案完全一致；
- `constexpr` 结果必须与标准答案完全一致。

对于语法错误的测试：

- 必须识别正确的 Missing / Extraneous Token；
- 必须给出正确的 0-indexed 行号；
- 输出中不得出现额外的 Parse Tree、ANTLR 默认错误文本或其他调试信息。

9.1 建议的实现顺序

建议按照以下顺序完成：

1. 先完成 `expression`，使用正确输入检查 Parse Tree；
2. 完成 `ConstExprVisitor.java`；
3. 完成 `Project2ErrorStrategy.java`；
4. 最后为形式参数 Missing `COMMA` 添加 Error Production；
5. 使用 `make test` 检查全部公开测试；
6. 自行补充边界测试后运行 `make handin`。